

School Assignment Sync

Snowflake Scheduler + GitHub Source of Record Setup & Handoff Guide

Start-to-finish build, operation, and handoff of the nightly
Asana school rep assignment sync.

DOCUMENT	Setup & Handoff Guide
VERSION	1.0 (Draft for handoff)
DATE	September 24, 2026
PREPARED BY	PMO Roger Cormier, Technical Project Manager
CURRENT MAINTAINER	Matt Larsen
CLASSIFICATION	Internal. Contains no credentials.

Contents

1 Overview	3
What this system does	3
Architecture	3
Operating model	4
2 Roles and Responsibilities	4
3 Configuration Worksheet	5
4 Prerequisites	5
5 Setup, Start to Finish	6
Phase 1: Build the repository	6
Phase 2: Create the service role and grants	8
Phase 3: Store Asana credentials in production	8
Phase 4: Connect Snowflake to the GitHub repository	9
Phase 5: First deployment and data migration	10
Phase 6: Scheduler and failure alert	10
Phase 7: Shadow week and go-live	12
Phase 8: Retire the sandbox	12
6 Operating Runbook	13
Daily check	13
Common operations	13
Changing the code	13
Credential rotation	14
7 Troubleshooting	14
8 Asana Behaviors the Receiving Team Must Know	15
9 Known Gaps at Handoff	15
10 Handoff Acceptance Checklist	16
A Object Inventory	17
B Configuration Settings Reference	18

1 Overview

What this system does

The School Assignment Sync keeps the rep role fields (BCM Rep, AC Rep, AP Rep, Payroll) on Asana monthly close projects aligned with the **School Assignment Sheet**, the Asana project that is the master record of who supports each school. Every night a Snowflake task compares intended state (the sheet) with actual state (the in-scope projects) and writes only the differences.

- **Reconciliation, not change detection.** Each run converges the downstream projects to the sheet. It is safe to re-run, heals after a failed run, and corrects manual drift. No webhooks or listeners are required for the nightly path.
- **Guardrails.** A mode switch (SHADOW logs only; LIVE writes), a kill switch (PAUSED), and a per-run write cap (MAX_WRITES) all live in a configuration table, not in code.
- **Bounded scope.** Only projects enabled in FANOUT_SCOPE are read or written.

Architecture

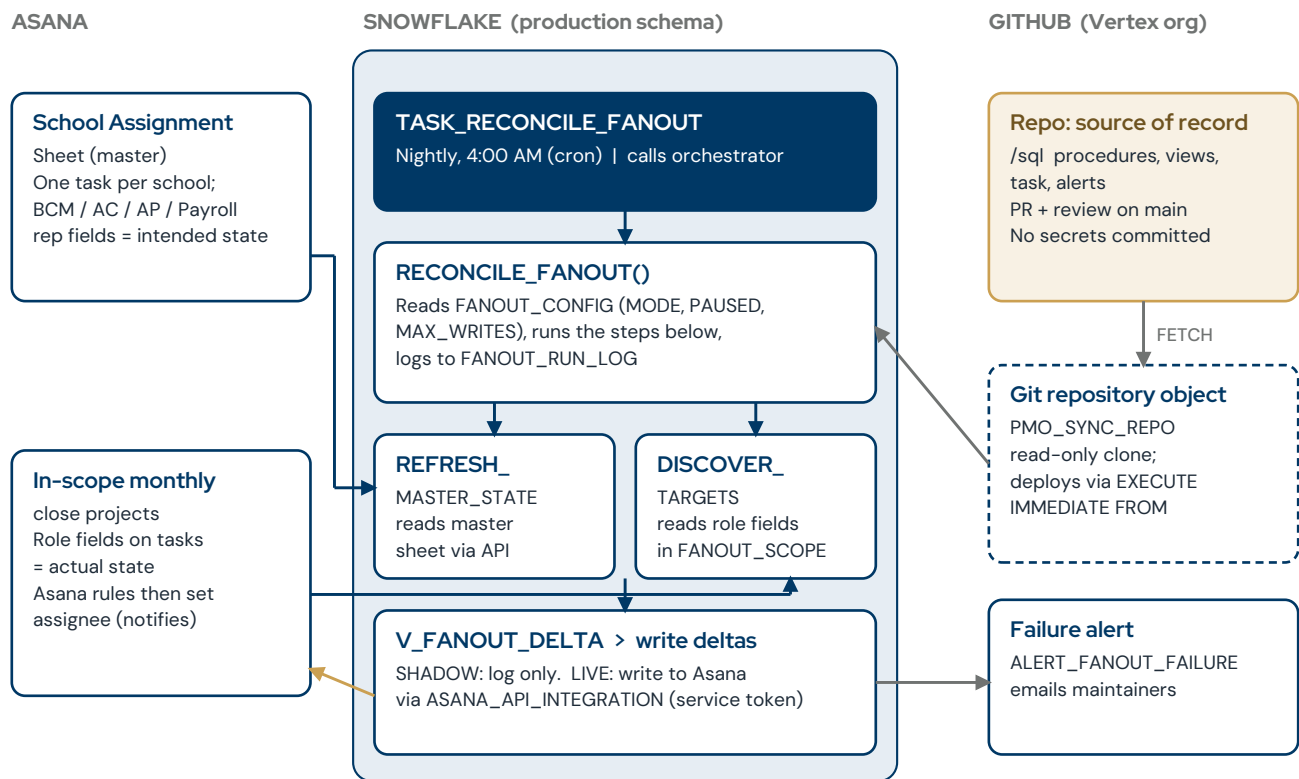


Figure 1. Data flows from the School Assignment Sheet into Snowflake, is compared with the in-scope projects, and only deltas are written back to Asana. Code flows one way: GitHub to Snowflake.

NOTE

The BI mirror of Asana in PROD_DB.ASANA is used for analysis and scope planning only. The runtime job calls the Asana API directly, because writes must go through the API and the mirror may lag.

Operating model

Concern	Where it lives	Why
Code (source of record)	Private repo asana-sas-automations in the Vertex GitHub org	Version history, pull-request review, and visibility beyond one person
Nightly schedule	Snowflake task	Logic already runs in Snowflake. No external scheduler, and no Snowflake credentials leave Snowflake
Runtime identity	Dedicated Snowflake service role and an Asana service token	Removes dependence on any individual's account
Deployment	Snowflake pulls from GitHub (read-only) and runs files with EXECUTE IMMEDIATE FROM	What runs is exactly what was reviewed and merged
Monitoring	Snowflake alert emails maintainers on failure	Failures surface without anyone checking manually

NOTE

GitHub Actions is intentionally **not** used to schedule the run. It would only trigger a procedure that already lives in Snowflake, and would require a key-pair service user, repository secrets holding a Snowflake private key, and UTC-only cron. GitHub's role here is the code repository and review gate.

2 Roles and Responsibilities

Names reflect the current team as of this document's date. The receiving team fills in its own column during handoff (Section 10).

Role	Responsibilities	Current	Receiving team
Business owner	Owns the School Assignment Sheet data and Asana monthly close rules; approves scope and go-live	Cora (Monthly Close process)	
PMO sponsor	Prioritization, sign-off on LIVE mode, handoff acceptance	Roger Cormier	
Maintainer	Code changes, deploys, daily run review, troubleshooting	Matt Larsen	
Snowflake administrator	Roles, grants, integrations, secrets requiring ACCOUNTADMIN	Vinay	
GitHub org / Asana admin	Repo access, service accounts, Asana service token	IT / Austin De Rossi	
BI / data platform	Production schema placement and PROD_DB access	BI team	

3 Configuration Worksheet

Decide every value below before running any step. All SQL in this guide uses these placeholders. Record final values here and in the repository README.

Placeholder	Meaning	Recommended / current	Final value
<PROD_DB> . <SCHEMA>	Production home for all sync objects	PROD_DB.WS4 (pending BI decision)	
<SYNC_ROLE>	Service role that owns objects, task, and alert	PMO_SYNC_ROLE	
<WAREHOUSE>	Warehouse for task and alert	COMPUTE_WH or a dedicated XS	
<OPERATORS>	People granted the service role to operate the sync	Maintainer + backup	
<GITHUB_ORG>	Vertex GitHub organization that hosts the repository	Confirm with IT	
<GITHUB_SVC_USER>	Non-personal GitHub account that owns the read-only token	Issued by IT	
<GITHUB_TOKEN>	Fine-grained token: read-only Contents on this repo only	Issued by IT; record expiry date	
<ASANA_SERVICE_TOKEN>	Org-sanctioned Asana token for the sync	Issued by Austin; replaces personal PAT	
<CRON>	Nightly run time and time zone	O 4 * * * America/Phoenix (confirm)	
<ALERT_RECIPIENTS>	Emails for failure alerts (must be verified Snowflake users)	Maintainer + backup	

IMPORTANT

Never commit tokens or passwords to the repository. Secrets are created directly in Snowflake by an administrator. The repo holds templates with placeholders only.

4 Prerequisites

Item	Evidence / owner
☐ Production schema confirmed (Section 3)	BI team / PMO sponsor
☐ Member access to the Vertex GitHub org for the maintainer	IT
☐ Service GitHub account and read-only fine-grained token issued	IT
☐ Asana service token issued with access to the School Assignment Sheet and in-scope projects	Austin De Rossi

Item	Evidence / owner
☐ Snowflake administrator available for Phases 2 through 4 (ACCOUNTADMIN or CREATE INTEGRATION)	Vinay
☐ Current SQL exported from the sandbox (MART_DB.BA_MLARSSEN): files 01 through 06	Maintainer
☐ Business owner aware that a shadow week precedes any live writes	PMO sponsor

NOTE

Starting point as of this document: all objects run in the sandbox schema `MART_DB.BA_MLARSSEN` under the maintainer's role and personal Asana token. A task was created on September 15, 2026 in SHADOW mode. This guide moves the system to production ownership.

5 Setup, Start to Finish

Complete the phases in order. Each phase ends with a checkpoint; do not proceed until it passes.

Phase	Outcome	Run by	Effort
1. Repository	SQL in GitHub, structured and safe to redeploy	Maintainer	2 to 3 hrs
2. Role and grants	Service role with required privileges	Snowflake admin	30 min
3. Asana credentials	Service token stored; external access allowed	Snowflake admin + maintainer	30 min
4. Connect GitHub	Snowflake can read the repo	Snowflake admin	30 min
5. First deploy	Objects built in production from the repo; data migrated	Maintainer	1 to 2 hrs
6. Scheduler and alert	Nightly task and failure email running	Maintainer	30 min
7. Shadow week, go-live	Validated behavior; LIVE writes enabled	Maintainer + sponsor	5+ nights
8. Retire sandbox	One scheduler, no personal credentials	Maintainer	30 min

Phase 1: Build the repository

1. Create (or confirm) the **private** repository `asana-sas-automations` in the Vertex org.
2. Unzip the starter file set into the repository (layout below). Then run `tools/export_sandbox_ddl.sql` in the sandbox and paste each exported definition into the file it names, replacing the TODO-EXPORT markers in `sql/02` through `sql/05`. The export already converts state tables to IF NOT EXISTS and strips sandbox qualifiers. Detailed steps: [runbooks/migration-from-sandbox.md](#).
3. Run `bash tools/check_repo.sh`. It fails on sandbox references, unfilled markers, unresolved placeholders, replaced state tables, missing RESUME lines, and likely secrets. Also read the pasted procedures for hardcoded GIDs or personal values.
4. Confirm state tables are safe to redeploy (see below).

5. Protect `main`: require a pull request, one approving review, code-owner review, and a passing `repo-checks` workflow. Fill in `.github/CODEOWNERS`.
6. Complete the README: configuration values (Section 3), owners, and credential expiry dates.

```
asana-sas-automations/
|-- README.md           status, config values, owners, credential expiry dates
|-- CHANGELOG.md        dated record of deploys, mode and config changes
|-- .gitignore
|-- .github/
|   |-- CODEOWNERS      required reviewers
|   |-- pull_request_template.md review checklist
|   |-- workflows/repo-checks.yml PR check only (no scheduling, no deploys)
|-- admin/              run once by an administrator; placeholders, NO secret values
|   |-- a1_role_and_grants.sql
|   |-- a2_asana_access.sql
|   |-- a3_github_integration.sql
|   |-- a4_notification_integration.sql
|   |-- a5_retire_sandbox.sql
|-- sql/                deployed in order with EXECUTE IMMEDIATE FROM
|   |-- 01_seed.sql      documentation only; never deployed
|   |-- 02_state_tables.sql config, scope, run log, user map tables (IF NOT EXISTS)
|   |-- 03_read_asana.sql LOAD_ASANA_USER_MAP, REFRESH_MASTER_STATE, DISCOVER_TARGETS
|   |-- 04_diff_write_rollback.sql delta/preview/unresolved views, RUN_FANOUT, ROLLBACK_FANOUT
|   |-- 05_orchestrator.sql RECONCILE_FANOUT
|   |-- 06_schedule.sql  TASK_RECONCILE_FANOUT
|   |-- 07_alerts.sql    ALERT_FANOUT_FAILURE
|-- runbooks/
|   |-- migration-from-sandbox.md first-time production setup
|   |-- deploy.md          every later change
|   |-- operations.md      daily check, operations, troubleshooting
|-- tools/
|   |-- export_sandbox_ddl.sql pulls current definitions out of the sandbox
|   |-- verify_deployment.sql post-deploy checks
|   |-- check_repo.sh       checks run by the PR workflow
|-- app/                 optional Streamlit console (not on the critical path)
-- docs/School_Assignment_Sync_Setup_Guide.pdf
```

Make state tables safe to redeploy. The original `06_schedule.sql` used `CREATE OR REPLACE TABLE FANOUT_CONFIG`. Rerunning it silently resets `MODE` to `SHADOW`, clears the kill switch, and restores the default write cap. Apply the same pattern to every table that holds data:

```
-- sql/02_state_tables.sql (excerpt)
CREATE TABLE IF NOT EXISTS FANOUT_CONFIG (
    SETTING VARCHAR(40), VALUE VARCHAR(200), NOTES VARCHAR(400));

INSERT INTO FANOUT_CONFIG (SETTING, VALUE, NOTES)
SELECT * FROM VALUES
    ('MODE', 'SHADOW', 'SHADOW = log only, no writes. LIVE = writes to Asana.'),
    ('MAX_WRITES', '200', 'Run aborts above this. Raise deliberately.'),
    ('PAUSED', 'FALSE', 'TRUE = skip everything. Kill switch.') AS v(s, val, n)
WHERE NOT EXISTS (SELECT 1 FROM FANOUT_CONFIG);

-- Repeat CREATE TABLE IF NOT EXISTS for FANOUT_SCOPE, FANOUT_RUN_LOG, ASANA_USER_MAP.
-- Views and procedures (files 03-05) may keep CREATE OR REPLACE: they hold no data.
```

CHECKPOINT

Repo contains files 02 through 07 with no hardcoded sandbox names (search for `BA_MLARSEN` returns nothing), no secret values, and branch protection is on.

Phase 2: Create the service role and grants

Run by the Snowflake administrator. Save as `admin/a1_role_and_grants.sql`.

```
USE ROLE SECURITYADMIN;
CREATE ROLE IF NOT EXISTS <SYNC_ROLE>;
GRANT ROLE <SYNC_ROLE> TO ROLE SYSADMIN;
GRANT ROLE <SYNC_ROLE> TO USER <OPERATOR_1>;
GRANT ROLE <SYNC_ROLE> TO USER <OPERATOR_2>;

USE ROLE ACCOUNTADMIN;
GRANT EXECUTE TASK ON ACCOUNT TO ROLE <SYNC_ROLE>;
GRANT EXECUTE ALERT ON ACCOUNT TO ROLE <SYNC_ROLE>;
GRANT USAGE ON WAREHOUSE <WAREHOUSE> TO ROLE <SYNC_ROLE>;
GRANT USAGE ON DATABASE <PROD_DB> TO ROLE <SYNC_ROLE>;
GRANT USAGE, CREATE TABLE, CREATE VIEW, CREATE PROCEDURE, CREATE TASK, CREATE ALERT,
      CREATE SECRET, CREATE NETWORK RULE, CREATE GIT REPOSITORY, CREATE STREAMLIT
      ON SCHEMA <PROD_DB>.<SCHEMA> TO ROLE <SYNC_ROLE>;
```

CHECKPOINT

An operator can run `USE ROLE <SYNC_ROLE>; USE SCHEMA <PROD_DB>.<SCHEMA>;` and `SHOW GRANTS TO ROLE <SYNC_ROLE>;` lists EXECUTE TASK and EXECUTE ALERT.

Phase 3: Store Asana credentials in production

The procedures call Asana through an external access integration named `ASANA_API_INTEGRATION` and a secret named `ASANA_PAT`. Integrations are account-level objects, and one with this name already serves the sandbox. Keep the same names so no procedure code changes: create the secret and network rule in the production schema, then extend the existing integration to allow them.

```
-- As <SYNC_ROLE> (the administrator enters the token value directly; never commit it)
USE ROLE <SYNC_ROLE>;
USE SCHEMA <PROD_DB>.<SCHEMA>;
CREATE OR REPLACE SECRET ASANA_PAT
  TYPE = GENERIC_STRING
  SECRET_STRING = '<ASANA_SERVICE_TOKEN>';

CREATE OR REPLACE NETWORK RULE ASANA_API_RULE
  MODE = EGRESS TYPE = HOST_PORT VALUE_LIST = ('app.asana.com');

-- As administrator: inspect, then extend the existing integration
USE ROLE ACCOUNTADMIN;
DESCRIBE EXTERNAL ACCESS INTEGRATION ASANA_API_INTEGRATION; -- note current rules/secrets
ALTER EXTERNAL ACCESS INTEGRATION ASANA_API_INTEGRATION SET
  ALLOWED_NETWORK_RULES = (<existing sandbox rule>, <PROD_DB>.<SCHEMA>.ASANA_API_RULE)
  ALLOWED_AUTHENTICATION_SECRETS = (MART_DB.BA_MLARSSEN.ASANA_PAT, <PROD_DB>.<SCHEMA>.ASANA_PAT);
GRANT USAGE ON INTEGRATION ASANA_API_INTEGRATION TO ROLE <SYNC_ROLE>;
```

IMPORTANT

Match the secret type and network rule to the sandbox definitions (`DESCRIBE SECRET`, `DESCRIBE NETWORK RULE`). The sandbox references are removed from the integration in Phase 8.

CHECKPOINT

`SHOW SECRETS IN SCHEMA <PROD_DB>.<SCHEMA>;` lists ASANA_PAT, and the integration description includes the production secret and rule.

Phase 4: Connect Snowflake to the GitHub repository

Requires ACCOUNTADMIN or CREATE INTEGRATION. Save a placeholder version as `admin/a3_github_integration.sql`.

```
USE ROLE ACCOUNTADMIN;
USE SCHEMA <PROD_DB>.<SCHEMA>;

-- 1. Read-only GitHub token (value entered directly, never committed)
CREATE OR REPLACE SECRET GITHUB_READ_SECRET
  TYPE = PASSWORD
  USERNAME = '<GITHUB_SVC_USER>'
  PASSWORD = '<GITHUB_TOKEN>';

-- 2. Allow Snowflake to reach the Vertex GitHub org
CREATE OR REPLACE API INTEGRATION VERTEX_GITHUB_INTEGRATION
  API_PROVIDER = git_https_api
  API_ALLOWED_PREFIXES = ('https://github.com/<GITHUB_ORG>/')
  ALLOWED_AUTHENTICATION_SECRETS = (GITHUB_READ_SECRET)
  ENABLED = TRUE;

GRANT USAGE ON INTEGRATION VERTEX_GITHUB_INTEGRATION TO ROLE <SYNC_ROLE>;
GRANT USAGE ON SECRET GITHUB_READ_SECRET TO ROLE <SYNC_ROLE>;

-- 3. Register the repository (as the service role, so it owns the object)
USE ROLE <SYNC_ROLE>;
CREATE OR REPLACE GIT REPOSITORY PMO_SYNC_REPO
  API_INTEGRATION = VERTEX_GITHUB_INTEGRATION
  GIT_CREDENTIALS = GITHUB_READ_SECRET
  ORIGIN = 'https://github.com/<GITHUB_ORG>/asana-sas-automations.git';

-- 4. Verify
ALTER GIT REPOSITORY PMO_SYNC_REPO FETCH;
SHOW GIT BRANCHES IN PMO_SYNC_REPO;
LIST @PMO_SYNC_REPO/branches/main/sql/;
```

NOTE

The Git repository object is a read-only clone. Snowflake never pushes to GitHub, so all edits happen in GitHub through pull requests.

CHECKPOINT

`LIST @PMO_SYNC_REPO/branches/main/sql/` returns files O2 through O7.

Phase 5: First deployment and data migration

This block is also the standard deploy procedure for every future change. Store it in `runbooks/deploy.md`.

```
USE ROLE <SYNC_ROLE>;
USE WAREHOUSE <WAREHOUSE>;
USE SCHEMA <PROD_DB>.<SCHEMA>;

ALTER GIT REPOSITORY PMO_SYNC_REPO FETCH;                -- pull latest main

EXECUTE IMMEDIATE FROM @PMO_SYNC_REPO/branches/main/sql/02_state_tables.sql;
EXECUTE IMMEDIATE FROM @PMO_SYNC_REPO/branches/main/sql/03_read_asana.sql;
EXECUTE IMMEDIATE FROM @PMO_SYNC_REPO/branches/main/sql/04_diff_write_rollback.sql;
EXECUTE IMMEDIATE FROM @PMO_SYNC_REPO/branches/main/sql/05_orchestrator.sql;
-- 06 and 07 are deployed in Phase 6 on first setup; include them in every later deploy.
```

Migrate data from the sandbox (first deploy only). Copy scope and the user map; do not copy FANOUT_CONFIG, so production starts in SHADOW.

```
-- Confirm column lists match before copying
INSERT INTO FANOUT_SCOPE SELECT * FROM MART_DB.BA_MLARSEN.FANOUT_SCOPE;
INSERT INTO ASANA_USER_MAP SELECT * FROM MART_DB.BA_MLARSEN.ASANA_USER_MAP;
-- Alternative for the user map: CALL LOAD_ASANA_USER_MAP();

-- Review scope: test projects must be disabled before any LIVE run
SELECT PROJECT_NAME, IS_ENABLED, NOTES FROM FANOUT_SCOPE ORDER BY PROJECT_NAME;
UPDATE FANOUT_SCOPE SET IS_ENABLED = FALSE WHERE PROJECT_NAME ILIKE '%Academy of Alameda%';
```

Validate manually in SHADOW before scheduling:

```
CALL REFRESH_MASTER_STATE();          -- return string reports schools missing an SFO Client
CALL DISCOVER_TARGETS();
SELECT * FROM V_UNRESOLVED_REPS;      -- reps on the sheet with no Asana user match
SELECT * FROM V_FANOUT_PREVIEW;      -- what LIVE mode would change
CALL RECONCILE_FANOUT();              -- full orchestrated run; MODE must read SHADOW
SELECT * FROM FANOUT_RUN_LOG ORDER BY 1 DESC LIMIT 20;
```

CHECKPOINT

Production results match a same-day sandbox run (change counts per project within expectation), the return string shows SHADOW, and every enabled project in scope is intentional.

Phase 6: Scheduler and failure alert

`sql/06_schedule.sql`:

```
CREATE OR REPLACE TASK TASK_RECONCILE_FANOUT
  WAREHOUSE = <WAREHOUSE>
  SCHEDULE = 'USING CRON <CRON>'          -- e.g. 'USING CRON 0 4 * * * America/Phoenix'
  COMMENT = 'Source of record: github.com/<GITHUB_ORG>/asana-sas-automations'
AS
  CALL RECONCILE_FANOUT();

ALTER TASK TASK_RECONCILE_FANOUT RESUME;  -- REQUIRED: CREATE OR REPLACE leaves the task suspended
```

IMPORTANT

Keep the **RESUME** line in the file. Without it, every redeploy quietly stops the nightly run.

Email notification integration (one-time, administrator). Recipients must be verified users in Snowflake:

```
USE ROLE ACCOUNTADMIN;
CREATE NOTIFICATION INTEGRATION IF NOT EXISTS PMO_SYNC_EMAIL
  TYPE = EMAIL ENABLED = TRUE
  ALLOWED_RECIPIENTS = (<ALERT_RECIPIENTS>);
GRANT USAGE ON INTEGRATION PMO_SYNC_EMAIL TO ROLE <SYNC_ROLE>;
```

sql/07_alerts.sql:

```
CREATE OR REPLACE ALERT ALERT_FANOUT_FAILURE
  WAREHOUSE = <WAREHOUSE>
  SCHEDULE = 'USING CRON 0 6 * * * America/Phoenix' -- two hours after the run
  IF (EXISTS (
    SELECT 1
    FROM (
      SELECT COUNT_IF(STATE IN ('FAILED', 'CANCELLED')) AS N_BAD,
             COUNT_IF(STATE = 'SUCCEEDED') AS N_OK
      FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
        TASK_NAME => 'TASK_RECONCILE_FANOUT',
        SCHEDULED_TIME_RANGE_START => DATEADD('hour', -24, CURRENT_TIMESTAMP())))
    )
    WHERE N_BAD > 0 OR N_OK = 0 -- failed, cancelled, or did not run at all
  ))
  THEN
    CALL SYSTEM$SEND_EMAIL('PMO_SYNC_EMAIL', '<ALERT_RECIPIENTS>',
      'School Assignment Sync: nightly run failed or did not run',
      'Check TASK_HISTORY and FANOUT_RUN_LOG. Runbook: runbooks/operations.md');
  ALTER ALERT ALERT_FANOUT_FAILURE RESUME;
```

OPEN ITEM

If **RECONCILE_FANOUT** handles an over-cap run by *returning* an abort message rather than raising an error, the task records **SUCCEEDED** and the alert does not fire. Either change the procedure to **RAISE** on abort, or add a condition on **RETURN_VALUE** with the exact abort text.

```
-- Deploy 06 and 07 from the repo, then verify
EXECUTE IMMEDIATE FROM @PMO_SYNC_REPO/branches/main/sql/06_schedule.sql;
EXECUTE IMMEDIATE FROM @PMO_SYNC_REPO/branches/main/sql/07_alerts.sql;
SHOW TASKS LIKE 'TASK_RECONCILE_FANOUT'; -- state = started
SHOW ALERTS LIKE 'ALERT_FANOUT_FAILURE'; -- state = started
EXECUTE TASK TASK_RECONCILE_FANOUT; -- optional immediate test run
```

CHECKPOINT

Task and alert both show **started**. Because the alert also fires when no run succeeded in 24 hours, a suspended task produces an email the next morning.

Phase 7: Shadow week and go-live

1. Leave MODE = SHADOW for at least five consecutive nightly runs.
2. Each morning, review the run history (query in Section 6). Look for failures, unexpected scope, and change counts that stay large night after night. Persistent large deltas indicate a comparison bug that would cause thousands of unnecessary writes in LIVE mode.
3. Resolve anything in V_UNRESOLVED_REPS and any schools missing an SFO Client match with the business owner.
4. Brief the business owner: once LIVE, Asana rules that watch the role fields will set assignees and send notifications. The sync write itself is silent; the rule reacting to it is not.
5. Obtain PMO sponsor sign-off, then switch to LIVE:

```
UPDATE FANOUT_CONFIG SET VALUE = 'LIVE' WHERE SETTING = 'MODE';
```

6. Verify the first LIVE run in Asana on several projects by eye. Record the switch date in the README change log.

CHECKPOINT

Five or more clean shadow runs reviewed, sponsor sign-off recorded, first LIVE run verified in Asana.

Phase 8: Retire the sandbox

```
-- Suspend the sandbox scheduler so two jobs can never run
ALTER TASK MART_DB.BA_MLARSSEN.TASK_RECONCILE_FANOUT SUSPEND;

-- Remove sandbox references from the shared integration (administrator)
ALTER EXTERNAL ACCESS INTEGRATION ASANA_API_INTEGRATION SET
  ALLOWED_NETWORK_RULES = (<PROD_DB>.<SCHEMA>.ASANA_API_RULE)
  ALLOWED_AUTHENTICATION_SECRETS = (<PROD_DB>.<SCHEMA>.ASANA_PAT);
```

- Revoke the maintainer's personal Asana PAT once no object references it.
- Keep sandbox objects for 30 days as a fallback, then drop them with the PMO sponsor's approval.
- Update the end-user SOP / help article to reference production ownership and the receiving team.

IMPORTANT

Never run the sandbox and production jobs in LIVE mode at the same time.

6 Operating Runbook

Daily check

```
USE ROLE <SYNC_ROLE>; USE SCHEMA <PROD_DB>.<SCHEMA>;
SELECT NAME, STATE, SCHEDULED_TIME, COMPLETED_TIME, RETURN_VALUE, ERROR_MESSAGE
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    TASK_NAME => 'TASK_RECONCILE_FANOUT', RESULT_LIMIT => 10))
ORDER BY SCHEDULED_TIME DESC;
```

RETURN_VALUE summarizes the run: mode, change count, and per-project counts.

Common operations

Need	Action
Run now (on demand)	EXECUTE TASK TASK_RECONCILE_FANOUT;
Stop all activity, keep schedule	UPDATE FANOUT_CONFIG SET VALUE='TRUE' WHERE SETTING='PAUSED'; Set back to FALSE to resume.
Stop the schedule itself	ALTER TASK TASK_RECONCILE_FANOUT SUSPEND; Use RESUME to restart.
Return to log-only	UPDATE FANOUT_CONFIG SET VALUE='SHADOW' WHERE SETTING='MODE';
Run aborted over write cap	Inspect V_FANOUT_PREVIEW first. Raise MAX_WRITES only after confirming the changes are intended.
Add or remove a project	Set IS_ENABLED in FANOUT_SCOPE. Preview in SHADOW before the next LIVE run.
Test one client safely	CALL RUN_FANOUT(TRUE, '<client>', 50); (dry run, one client, capped). Check the procedure header for parameters.
New rep does not resolve	Check V_UNRESOLVED_REPS, then CALL LOAD_ASANA_USER_MAP();
Undo a bad run	Use ROLLBACK_FANOUT per its procedure header, scoped to the affected run. Confirm with the business owner first.

Changing the code

1. Create a branch and edit files in GitHub (or locally). Never edit procedures directly in a Snowsight worksheet: if it is not in `main`, it is not the record.
2. Open a pull request describing the change and expected effect on run output. A second person reviews and approves.
3. Merge to `main`.
4. In Snowflake, run the deploy block (Phase 5) including files O6 and O7.
5. Confirm the task is still `started` and review the next run's output.

Credential rotation

```
-- Asana service token (administrator or <SYNC_ROLE>)
ALTER SECRET <PROD_DB>.<SCHEMA>.ASANA_PAT SET SECRET_STRING = '<new token>';

-- GitHub read-only token: rotate before its expiry date (recorded in README)
ALTER SECRET <PROD_DB>.<SCHEMA>.GITHUB_READ_SECRET SET PASSWORD = '<new token>';
ALTER GIT REPOSITORY PMO_SYNC_REPO FETCH;          -- confirms the new token works
```

NOTE

Fine-grained GitHub tokens expire. Put the expiry date in the README and a calendar reminder two weeks ahead for the maintainer and backup.

7 Troubleshooting

Symptom	Likely cause	Fix
No runs after a deploy	CREATE OR REPLACE TASK left the task suspended	<code>ALTER TASK TASK_RECONCILE_FANOUT RESUME</code> ; Confirm 06 ends with RESUME.
RESUME fails with insufficient privileges	EXECUTE TASK not granted to the role	Administrator grants EXECUTE TASK ON ACCOUNT (Phase 2)
MODE back to SHADOW, scope reset	A state table was recreated on deploy	Restore from FANOUT_RUN_LOG and records; convert the table to IF NOT EXISTS
FETCH or LIST fails with auth error	GitHub token expired, revoked, or lacks Contents: read	Issue a new token; <code>ALTER SECRET GITHUB_READ_SECRET</code>
Asana 401 / 403 in run log	Asana service token revoked or missing project access	Rotate ASANA_PAT; confirm token access to sheet and projects
Write fails for specific schools	SFO Client value on the sheet does not match the project's dropdown option (LLC or alias mismatch)	Correct the value with the business owner; rerun
Rep never assigned	Rep email has no Asana user match	Check V_UNRESOLVED_REPS; run <code>LOAD_ASANA_USER_MAP</code>
Invalid identifier (e.g. BCM)	Query uses display label instead of column name	Use actual columns (e.g. BCM_CONTACT); check INFORMATION_SCHEMA.COLUMNS
Run aborted, no writes	Delta exceeded MAX_WRITES	Inspect V_FANOUT_PREVIEW before raising the cap
No failure email	Recipient unverified, or abort returned SUCCEEDED	Verify recipient emails; see Phase 6 open item
Users flooded with notifications	Asana rules set assignee when role fields change	Expected on first LIVE run; point users to the SOP section on reducing notifications
Object not found after migration	Hardcoded sandbox name in a procedure	Search repo for BA_MLARSEN; fix, merge, redeploy

8 Asana Behaviors the Receiving Team Must Know

- **The sync is silent; the rules are not.** Writing a role field sends no notification. The business owner's monthly close rules react to the change and set the task assignee, which does notify. A rep newly responsible for a school receives a batch on the first LIVE run.
- **Role fields are discovered per project, not hardcoded.** `DISCOVER_TARGETS` resolves each project's own field IDs. A project whose fields are missing is skipped rather than written incorrectly.
- **AP Rep is the standard field.** The legacy AP Representative field was retired across monthly close projects. Removing a field in Asana deletes its values, so field changes should be coordinated with the maintainer.
- **SFO Client is the join key.** Schools route to projects by matching the SFO Client value. Alias or LLC naming differences are the most common cause of skipped schools.
- **Scope is explicit.** Nothing outside `FANOUT_SCOPE` is read or written, which bounds the impact of any defect.

9 Known Gaps at Handoff

Status as of September 24, 2026. Each item should be closed or explicitly accepted by the receiving team.

Item	Status	Owner
Production schema and service role decision	Pending	PMO sponsor / BI
Asana service token replacing personal PAT	Requested; not confirmed	Austin De Rossi
Vertex GitHub org access and service token	IT request submitted	IT
Sandbox task (Sept 15) resume and shadow results	Unconfirmed whether RESUME succeeded or history was reviewed	Maintainer
Abort reporting may not trigger the alert	Open (Phase 6)	Maintainer
Test project Academy of Alameda enabled in scope	Must be disabled before LIVE	Maintainer
On-demand trigger for business users (Asana external action)	Not built; separate design item needing an endpoint	Maintainer / business owner
Sandbox object names and signatures in Appendix A	Confirm against the live sandbox before migration	Maintainer

10 Handoff Acceptance Checklist

The receiving team signs off only when every item is complete.

Item	Evidence / owner
☐ Receiving team members have repo access and the role	Screenshot / SHOW GRANTS
☐ Configuration worksheet (Section 3) fully completed in README	README link
☐ Receiving maintainer performed one code change end to end (PR, merge, deploy)	PR link
☐ Receiving maintainer ran the daily check and explained a RETURN_VALUE	Walkthrough date
☐ Pause, resume, and SHADOW/LIVE switch demonstrated	Walkthrough date
☐ Alert recipients updated to receiving team and a test alert received	Email received
☐ Token expiry dates recorded with calendar reminders	README
☐ All items in Section 9 closed or accepted in writing	Sign-off
☐ Section 2 contacts updated	This document / README

Handing-off maintainer

Receiving maintainer

PMO sponsor

Name / date

Name / date

Name / date

A Object Inventory

Confirm names against the sandbox with `SHOW OBJECTS IN SCHEMA MART_DB.BA_MLARSEN;` and `SHOW PROCEDURES IN SCHEMA MART_DB.BA_MLARSEN;` before migrating.

Object	Type	Purpose	File
FANOUT_CONFIG	Table	MODE, PAUSED, MAX_WRITES settings	02
FANOUT_SCOPE	Table	In-scope projects and IS_ENABLED flag	02
FANOUT_RUN_LOG	Table	Per-run and per-write log, used for audit and rollback	02
ASANA_USER_MAP	Table	Rep email to Asana user ID	02
REFRESH_MASTER_STATE	Procedure	Reads School Assignment Sheet into intended state	03
DISCOVER_TARGETS	Procedure	Reads role fields on in-scope projects (actual state)	03
LOAD_ASANA_USER_MAP	Procedure (Python)	Refreshes the user map from Asana	03
V_FANOUT_DELTA	View	Intended vs. actual differences to write	04
V_FANOUT_PREVIEW	View	Human-readable preview of pending changes	04
V_UNRESOLVED_REPS	View	Reps with no Asana user match	04
RUN_FANOUT	Procedure	Writes deltas (dry-run flag, client filter, cap)	04
ROLLBACK_FANOUT	Procedure	Reverts writes from a logged run	04
RECONCILE_FANOUT	Procedure	Orchestrates a full run using FANOUT_CONFIG	05
TASK_RECONCILE_FANOUT	Task	Nightly schedule	06
ALERT_FANOUT_FAILURE	Alert	Emails on failed runs	07
PMO_SYNC_REPO	Git repository	Read-only clone of the GitHub repo	admin
ASANA_PAT, GITHUB_READ_SECRET	Secrets	Asana service token; GitHub read token	admin
ASANA_API_RULE	Network rule	Egress to app.asana.com	admin
ASANA_API_INTEGRATION, VERTEX_GITHUB_INTEGRATION, PMO_SYNC_EMAIL	Integrations (account-level)	Asana egress, GitHub access, email	admin

B Configuration Settings Reference

Setting	Values	Effect	Guidance
MODE	SHADOW LIVE	SHADOW computes and logs every change without writing. LIVE writes to Asana.	Start every new environment and major change in SHADOW
PAUSED	TRUE FALSE	TRUE skips the run entirely and returns immediately.	Kill switch; faster than suspending the task
MAX_WRITES	Integer (default 200)	Run aborts without writing if the delta exceeds this.	Raise deliberately, after previewing changes